

Lab4: Recursion

COCS-221: Computer Science B



Objective

By the end of this lab, you will be able to

- Identify base cases and solve smaller problems recursively.
- Count and search through nested lists using recursion.
- Use a recursive helper function to check palindromes.

Exercise

Getting started: Nested gift boxes

A box is a Python list. Each item is a gift (a string) or another box (a list). A box may be empty or contain several levels of inner boxes.

```
box = ["book", ["pen", "pencil"],  
      [["toy"], "watch"]]
```

This example contains five gifts. Inner boxes are containers, not gifts. Draw the boxes and explain why `len(box)` does not give the total number of gifts.

Use `isinstance(item, list)` to check whether an item is an inner box. Loops may inspect items in a box; recursion must process inner boxes. Do not modify the input lists.

Use Python 3. Assume all box inputs contain only strings and lists. No extra packages or input validation are required.

Exercise 1: Count all gifts

Write a recursive function `count_gifts(box)` that returns the total number of gifts in the box and all its inner boxes.

- Return an integer. An empty box contains zero gifts.
- Count repeated gift names separately.
- Call `count_gifts` recursively for each inner box.

Test your function with the following inputs. The comments show the expected results:

```
count_gifts([]) # 0
count_gifts(["book"]) # 1
count_gifts(box) # 5
count_gifts([], ["ball",
                ["kite", "ball"]]) # 3
count_gifts([["ring"]]) # 1
```

Hint: Each item contributes either one gift or the result of counting the gifts in an inner box.

Exercise 2: Search for a gift

Write a recursive function `contains_gift(box, target)` that returns `True` if the target gift is anywhere in the box, otherwise `False`. The target is a string.

- Match the full gift name. "Toy" and "toy" are different.
- Search inner boxes recursively. Stop when a match is found; return `False` only after all items have been checked.

```
contains_gift(box, "toy") # True
contains_gift(box, "watch") # True
contains_gift(box, "phone") # False
contains_gift(box, "Toy") # False
contains_gift([], "book") # False
contains_gift([], ["pen"], "pen") # True
```

Hint: An unsuccessful search in one inner box does not mean the target is absent from the remaining boxes.

Exercise 3: Palindrome numbers

A palindrome reads the same forwards and backwards. For example, 12321 is a palindrome, while 12341 is not.

Write `isPalindromeNum(n)` for a non-negative integer. Convert `n` to a string, then use a recursive helper function to compare the characters at the left and right positions.

```
def isPalindromeNum(n):
    digits = str(n)
    def check(left, right):
        # TODO: stopping cases and recursion
        pass
    return check(0, len(digits) - 1)
```

- Return a Boolean, not the strings "True" or "False".
- Move the two positions toward the centre. Do not use loops, `reversed()`, or reverse slicing such as `[::-1]` to perform the palindrome check.
- Leading zeros are not part of the integer input.

Test each of the following numbers:

0, 7, 11, 123454321, 1357531, 168861

Expected: True for each number

10, 112221 # Expected: False for each

Hint: What happens when the positions meet or cross? What happens when the two characters differ?

Testing and reflection

Run every required test and add one test per core exercise. Print labelled results in your test code; functions must return their answers.

Answer in comments: What stops each function? What becomes smaller? Why can returning False too early fail? Explain one extra test and the mistake it could detect.

Challenge Exercise 1: Unpack all gifts

Write a recursive function `unpack(box)` that returns a new, flat list of gift names. Keep their left-to-right order and duplicates. Do not change the original box or use a library to flatten it.

```
unpack(box)
# ["book", "pen", "pencil", "toy", "watch"]
unpack(["ball", "ball"]) # ["ball", "ball"]
unpack([]) # []
```

Challenge Exercise 2: Find the deepest box

Write a recursive function `box_depth(box)` that returns the greatest number of box levels. Count the outer box as level 1, even when empty. Gifts do not add a level.

```
box_depth([]) # 1
box_depth(["book", "pen"]) # 1
box_depth(["book"]) # 2
box_depth(["book", ["toy"]]) # 3
box_depth([[]]) # 2
```

Both challenges are optional. Complete the core exercises first.